

Architecture\_md

# AI Test Case Generator — Solution Architecture (Learner Edition)

**Document status:** Proposed architecture (finalized from a 20-question MCQ session)

**Target audience:** A single developer building this as a learning project

**Deployment:** Local / self-hosted (runs on your machine)

**Backend:** Node.js + TypeScript + Express

**Frontend:** React + TypeScript

**Database + vector store:** MongoDB Atlas free tier (M0) — native Vector Search + full-text (BM25)

**Embeddings:** Configurable adapter, default Mistral AI (local model as free fallback)

**LLM:** Configurable adapter — OpenAI / Groq / Anthropic, default Groq (local Ollama optional)

---

## 1. Purpose

This application ingests requirement documents and API specifications, retrieves the most relevant evidence using hybrid search, and generates structured, traceable test cases with a confidence score. It is deliberately scoped as a **learning project**: it teaches the core Retrieval-Augmented Generation (RAG) pipeline — ingestion, chunking, embeddings, hybrid retrieval, and validated LLM generation — while avoiding enterprise plumbing (SSO, multi-tenant RBAC, external connectors) that adds complexity without adding learning value.

The design keeps a clean upgrade path: every "learner" decision has a documented route to its enterprise-grade counterpart.

---

## 2. Confirmed Architecture Decisions

These are the finalized answers from the 20-question architecture questionnaire.

| # | Area               | Decision                                                         |
|---|--------------------|------------------------------------------------------------------|
| 1 | Deployment         | Local / self-hosted; app code runs on your machine               |
| 2 | Users & tenancy    | Single user, no login; data separated by `projectId`             |
| 3 | Input sources      | File uploads (PDF, Excel, Markdown/text) + Swagger/OpenAPI specs |
| 4 | Database & vectors | MongoDB Atlas M0 free tier (Vector Search + full-text)           |
| 5 | Embeddings         | Configurable adapter, default Mistral, local model fallback      |
| 6 | LLM generation     | Configurable adapter (OpenAI/Groq/Anthropic), default Groq       |
| 7 | Chunking           | Structure-aware + semantic boundaries + overlap                  |
| 8 | Ingestion mode     | In-process async with job IDs and status polling                 |

| 9 | Retrieval | Hybrid BM25 + vector with score fusion |  
| 10 | Query preprocessing | Normalization + abbreviation/synonym expansion (dictionary) |  
| 11 | Context assembly | Deduplicate + top-K + token cap (summarization later) |  
| 12 | Output format | Rich schema + strict JSON Schema validation + one repair retry |  
| 13 | Generation controls | Test types, coverage level, priority, source filters, format |  
| 14 | Review workflow | Single-step: draft → approved/rejected, with edit |  
| 15 | Confidence score | Composite score from signals → High/Medium/Low bands |  
| 16 | Traceability | Basic evidence citations preserved end-to-end |  
| 17 | Export | JSON + CSV/Excel download |  
| 18 | Observability | Structured logging + correlation/request IDs |  
| 19 | Config & secrets | `.env` + validated typed config (fail fast on missing keys) |  
| 20 | Voice chat | Documented future extension, not built now |

---

### ## 3. Supported Inputs

The ingestion layer supports two structurally different source families (chosen to teach two chunking strategies):

- **Documents:** PDF, Excel (`.xlsx`/`.csv`), and Markdown/plain text. These carry requirements, BRDs, user stories, and notes.
- **API specifications:** Swagger / OpenAPI (JSON or YAML). These carry endpoints, parameters, request/response schemas, and examples.

Each ingested item is normalized into a common record shape (see §7.2) regardless of its source type, so the rest of the pipeline treats everything uniformly.

> **Upgrade path:** Additional enterprise sources (Jira, ADO, Confluence, Git, defect DBs, recordings) can be added later behind a common connector interface without changing the retrieval/generation core.

---

### ## 4. High-Level Architecture

```mermaid

flowchart TB

```
UI["React + TypeScript Web App"] --> API["Express API (/api/v1)"]
API --> CFG["Typed Config + .env"]
API --> JOBS["In-process Async Jobs (job IDs)"]
JOBS --> ING["Ingestion Pipeline"]
API --> RET["Retrieval + Generation Pipeline"]
ING --> EMB["Embedding Adapter (Mistral / local)"]
```

```
RET --> EMB
RET --> LLM["LLM Adapter (Groq / OpenAI / Anthropic)"]
ING --> DB["MongoDB Atlas M0"]
RET --> DB
API --> LOG["Structured Logging + Correlation IDs"]
...
```

Everything runs locally as a small number of processes:

- One **Express API** process (also runs the in-process ingestion jobs).
- One **React dev server** (frontend).
- **MongoDB Atlas M0** in the cloud (free), reached over the network — this is the only non-local piece, and it exists purely to give you native hybrid search without installing a vector DB.

---

## ## 5. Local Deployment Design

| Component      | How it runs locally                                                   | Responsibility                               |
|----------------|-----------------------------------------------------------------------|----------------------------------------------|
| Frontend       | <code>`vite` / `react` dev server (<code>`npm run dev`</code>)</code> | UI for upload, generation, review, export    |
| API            | Node + Express ( <code>`npm run dev`</code> )                         | REST API, validation, job orchestration      |
| Ingestion jobs | Background tasks inside the API process                               | Parse → chunk → embed → store                |
| Database       | MongoDB Atlas M0 (free cloud tier)                                    | Documents, chunks, vectors, jobs, test cases |
| Config/secrets | <code>`.env`</code> + typed config module                             | API keys and settings, validated at startup  |
| Logs           | stdout (structured JSON via pino/winston)                             | Debugging and request tracing                |

No Docker, no Kubernetes, no message broker required to start. The async job model (§8) is designed so you can later move ingestion into a separate worker + queue without changing the API contract.

---

## ## 6. Core Components

### ### 6.1 React Web Application

- Project selector (switch active ``projectId``).
- File upload (PDF/Excel/Markdown) and Swagger/OpenAPI upload.
- Ingestion job status view (polls by job ID).
- Test-generation form with controls (test types, coverage, priority, source filters, format).
- Results view: generated test cases with confidence band and evidence citations.
- Single-step review: edit, approve, or reject.
- Export button (JSON / CSV / Excel).

### ### 6.2 Express API

- Versioned REST routes under `/api/v1`.
- Request validation (Zod) and consistent error envelopes.
- Correlation/request ID middleware for tracing.
- Creates ingestion and generation jobs; reports status.
- Enforces `projectId` scoping on every query.

### ### 6.3 Ingestion Pipeline

- Validates file type/size before parsing.
- Extracts content (PDF text, Excel rows/sheets, Markdown, Swagger operations).
- Applies structure-aware + semantic chunking with overlap.
- Computes a content hash for each chunk (dedup + change detection).
- Calls the embedding adapter and upserts chunks + vectors into MongoDB.

### ### 6.4 Embedding Adapter

- Single interface: `embed(texts: string[]): number[][]` plus a `dimension` property.
- Default implementation: **Mistral** (`mistral-embed`).
- Optional free/offline implementation: a local model (e.g., `all-MiniLM-L6-v2`).
- The active adapter's `dimension` must match the MongoDB vector index dimension. Switching providers requires re-embedding and rebuilding the index.

### ### 6.5 Retrieval & Generation Pipeline

- Query preprocessing (normalize + abbreviation/synonym expansion).
- Hybrid BM25 + vector search with score fusion.
- Deduplication of near-identical chunks + top-K selection + token cap.
- Prompt assembly (instructions + query + cited evidence + generation controls).
- LLM call via the configurable adapter.
- Strict JSON Schema validation with one bounded repair retry.
- Composite confidence scoring and citation preservation.

### ### 6.6 LLM Adapter

- Single interface: `generate(prompt): StructuredResult`.
- Adapters for **Groq** (default), **OpenAI**, and **Anthropic**; optional local **Ollama**.
- Timeouts and a bounded retry; no dimension lock-in, so models are swappable via config.

### ### 6.7 Review Component

- Simple state machine: `draft → approved` / `draft → rejected`, plus in-place edits that create a new version.
- Stores reviewer comment and timestamp.

### ### 6.8 Export Component

- Serializes approved (or selected) test cases to JSON and CSV/Excel.
- Preserves evidence citations in the export.

---

## ## 7. Ingestion Pipeline (Detail)

```
```mermaid
flowchart TD
    U["Upload: PDF / Excel / Markdown / Swagger"] --> V["Validate type & size"]
    V --> X["Extract content (source-specific)"]
    X --> N["Normalize into common record + metadata"]
    N --> C["Structure-aware + semantic chunking + overlap"]
    C --> H["Content hash per chunk"]
    H --> E["Embedding adapter (Mistral / local)"]
    E --> M["Upsert chunks + vectors into MongoDB"]
    M --> J["Update ingestion job status"]
```
```

### ### 7.1 Processing Mode

- The API creates an `ingestion\_jobs` record and returns `202 Accepted` with a `jobId`.
- Work runs as a background task inside the Node process (`queued` → processing → done/failed).
- The client polls `GET /ingestions/:jobId` for progress.
- Jobs record counts, warnings, and errors so failures are visible.

> **Upgrade path:** replace the in-process task with a Pub/Sub (or BullMQ/Redis) worker later — the API contract (`jobId` + polling) does not change.

### ### 7.2 Normalized Record

Every source becomes a common structure:

- `projectId`, `sourceType` (`pdf`/`excel`/`markdown`/`openapi`), `sourceId`, `sourceName`.
- `title`, `rawText`, `normalizedText`, `contentType`.
- `version`, `checksum`, `createdAt`, `updatedAt`.
- `tags`, optional `release`/`component`, and any extracted `requirementIds`.

### ### 7.3 Chunking Strategy (Structure-aware + Semantic + Overlap)

- **Markdown/PDF:** split by headings/sections first; if a section is too long, split further at semantic/paragraph boundaries; add controlled overlap.
- **Excel:** split by sheet, then by logical table / row-group boundaries.
- **Swagger/OpenAPI:** one chunk per **operation** (`METHOD /path`) including its parameters, request body schema, responses, and examples — structure is the meaning here.
- Every chunk keeps a pointer back to its exact source location for citations.

---

## ## 8. Retrieval & Generation Pipeline (Detail)

```mermaid

flowchart TD

```
Q["User query + generation controls"] --> P["Normalize + expand abbreviations/synonyms"]
P --> S["Hybrid BM25 + vector search (scoped by projectId)"]
S --> F["Score fusion"]
F --> D["Deduplicate + top-K + token cap"]
D --> A["Assemble prompt (query + cited evidence + controls)"]
A --> L["LLM adapter (Groq / OpenAI / Anthropic)"]
L --> Vld["Validate against JSON Schema (1 repair retry)"]
Vld --> Sc["Composite confidence score"]
Sc --> R["Store draft test cases with citations"]
```

```

### ### 8.1 Retrieval Controls

- **Mandatory filter:** `projectId` (enforced server-side; never trusted from the client body alone).
- **Optional filters:** source type, release/component, tags, date.
- Search returns each candidate's source citation and relevance score.
- If retrieval quality is very low (few/weak matches), the app warns the user rather than generating unsupported test cases.

### ### 8.2 Query Preprocessing

- Normalize: lowercase, trim, strip noise punctuation.
- Expand using a small domain dictionary, e.g. `{ "auth": ["authentication", "login", "sign-in"], "ac": ["acceptance criteria"], "reg": ["regression"] }`.

> **Upgrade path:** add optional LLM-based query rewriting later.

### ### 8.3 Context Assembly

- Remove near-duplicate chunks (content hash / similarity threshold).
- Keep top-K and enforce a token/character budget.
- Preserve each included chunk's citation.

> **Upgrade path:** add summarization/compression of retrieved context to fit more evidence (target end-state).

### ### 8.4 Generated Test-Case Schema

Each test case is generated as JSON validated against a strict schema:

```json

```
{
  "testCaseId": "TC-checkout-001",
```

```

"title": "Successful checkout with valid card",
"objective": "Verify a logged-in user can complete checkout",
"preconditions": ["User is authenticated", "Cart has 1+ items"],
"testData": { "card": "valid test card" },
"steps": [
  { "step": 1, "action": "Open cart", "expectedResult": "Cart page shows items" },
  { "step": 2, "action": "Click checkout", "expectedResult": "Payment page opens" }
],
"testType": "functional",
"priority": "high",
"classification": "positive",
"citations": [{ "chunkId": "chunk-123", "sourceName": "checkout.md" }],
"assumptions": ["Payment gateway is available"],
"confidence": { "score": 0.82, "band": "High" },
"generatedBy": { "provider": "groq", "model": "llama-3.x", "promptVersion": "v1" },
"review": { "state": "draft" }
}
...

```

Validation uses Zod/Ajv. On a validation failure, the pipeline sends the errors back to the LLM once for a bounded repair; if it still fails, the generation is marked `failed` (no broken data is stored).

---

## ## 9. LLM & Embedding Provider Strategy

Both AI layers use a **provider adapter** so the pipeline never hard-codes a vendor.

- **Embeddings:** default Mistral; local model as a free/offline fallback. Vector index dimension is a single config value tied to the active adapter.
- **Generation:** default Groq (free/fast for iteration); OpenAI and Anthropic swappable via config; optional local Ollama.
- Each adapter handles its own timeout and one bounded retry.
- No silent cross-provider fallback for restricted data — provider is an explicit config choice.

---

## ## 10. Confidence Score

A transparent **composite** score (not the LLM's self-reported number, which is unreliable).

| Signal | Suggested weight | Source |
|--------|------------------|--------|
| ---    | ---              | ---    |

| Retrieval / fusion relevance quality | 40% | Search scores of the evidence used |  
| Requirement coverage | 30% | How many requested requirements are addressed |  
| Output completeness | 30% | Required schema fields present and non-empty |

Bands:

- **High:** `>= 0.80``
- **Medium:** `0.60 – 0.79``
- **Low:** `< 0.60``

The score prioritizes which test cases a human should review first; it is not a proof of correctness.

> **Upgrade path:** calibrate weights/thresholds against real reviewer decisions once usage data exists.

---

## ## 11. Review Workflow (Single-step)

```
```mermaid
stateDiagram-v2
    [*] --> Draft
    Draft --> Draft: Edit (new version)
    Draft --> Approved: Approve
    Draft --> Rejected: Reject (with comment)
```
```

Each transition stores actor comment and timestamp. Edits create a new version so history is preserved.

> **Upgrade path:** add a second approval stage (tester → lead), immutable audit events, and stale-approval version checks.

---

## ## 12. Data Model (MongoDB Collections)

| Collection | Purpose |  
|---|---|  
| ``projects`` | Project settings and the active-project list |  
| ``source_documents`` | Normalized source metadata, versions, checksums |  
| ``chunks`` | Chunk text, metadata, embedding vector, content hash |  
| ``ingestion_jobs`` | Job state, progress, counts, errors |  
| ``generation_requests`` | Query, controls, execution metadata |



| `test\_cases` | Current test-case records (with review state + citations) |  
| `test\_case\_versions` | Prior versions after edits |  
| `abbreviations` | Domain dictionary for query expansion |

#### ### Required Indexes

- **\*\*Full-text (BM25)\*\*** search index on chunk text.
- **\*\*Vector Search\*\*** index matching the active embedding adapter's dimension and similarity metric.
- Compound indexes beginning with `projectId` on `chunks`, `source\_documents`, `test\_cases`.
- Unique index on `(projectId, sourceId, version)` and on chunk `contentHash` for dedup.

---

### ## 13. REST API Design

Base path: `/api/v1`. All list/query endpoints require a `projectId` and are scoped server-side.

#### ### 13.1 Projects

| Method | Endpoint             | Function                      |
|--------|----------------------|-------------------------------|
| GET    | /projects            | List projects                 |
| POST   | /projects            | Create a project              |
| GET    | /projects/:projectId | Read project settings         |
| PATCH  | /projects/:projectId | Update project settings       |
| DELETE | /projects/:projectId | Delete a project and its data |

#### ### 13.2 Uploads & Ingestion

| Method | Endpoint                      | Function                                                       |
|--------|-------------------------------|----------------------------------------------------------------|
| POST   | /uploads                      | Upload a file (PDF/Excel/Markdown/Swagger); returns `uploadId` |
| POST   | /ingestions                   | Start ingestion for an upload; returns `202` + `jobId`         |
| GET    | /ingestions/:jobId            | Job status, counts, warnings, errors                           |
| POST   | /ingestions/:jobId/retry      | Retry a failed ingestion job                                   |
| GET    | /source-documents             | List ingested sources (by project)                             |
| GET    | /source-documents/:documentId | Source metadata + versions                                     |
| DELETE | /source-documents/:documentId | Delete a source and its chunks                                 |

#### ### 13.3 Retrieval & Evidence

| Method | Endpoint           | Function                                    |
|--------|--------------------|---------------------------------------------|
| POST   | /retrieval/search  | Run scoped hybrid retrieval (debug/preview) |
| GET    | /evidence/:chunkId | Fetch a source chunk + citation             |

#### ### 13.4 Generation & Test Cases

| Method | Endpoint                                            | Function                                                            |
|--------|-----------------------------------------------------|---------------------------------------------------------------------|
| POST   | <code>/generations`</code>                          | Create a generation request; returns <code>`jobId`</code> or result |
| GET    | <code>/generations/:generationId`</code>            | Status, controls, result summary                                    |
| POST   | <code>/generations/:generationId/regenerate`</code> | Regenerate with feedback/changes                                    |
| GET    | <code>/generation-options`</code>                   | Supported test types, coverage levels, formats                      |
| GET    | <code>/test-cases`</code>                           | List/filter test cases (by project)                                 |
| GET    | <code>/test-cases/:testCaseId`</code>               | Read a test case + current version                                  |
| PATCH  | <code>/test-cases/:testCaseId`</code>               | Edit a draft (creates a version)                                    |
| GET    | <code>/test-cases/:testCaseId/versions`</code>      | Version history                                                     |
| GET    | <code>/test-cases/:testCaseId/confidence`</code>    | Confidence score + signal breakdown                                 |

Example generation request:

```
```json
{
  "projectId": "proj-001",
  "query": "Generate checkout test cases for release 4.2",
  "sourceFilters": { "release": "4.2", "sourceType": ["markdown", "openapi"] },
  "testTypes": ["functional", "api", "negative"],
  "coverageLevel": "comprehensive",
  "priorities": ["high", "medium"],
  "outputFormat": "structured"
}
```
```

### ### 13.5 Review

| Method | Endpoint                                       | Function                       |
|--------|------------------------------------------------|--------------------------------|
| POST   | <code>/test-cases/:testCaseId/approve`</code>  | Approve a draft (with comment) |
| POST   | <code>/test-cases/:testCaseId/reject`</code>   | Reject a draft (with comment)  |
| POST   | <code>/test-cases/:testCaseId/comments`</code> | Add a review comment           |

### ### 13.6 Export

| Method | Endpoint                         | Function                                                               |
|--------|----------------------------------|------------------------------------------------------------------------|
| POST   | <code>/exports`</code>           | Create an export (JSON / CSV / Excel); returns <code>`exportId`</code> |
| GET    | <code>/exports/:exportId`</code> | Export status + download link                                          |

### ### 13.7 Config & Health

| Method | Endpoint                        | Function                                      |
|--------|---------------------------------|-----------------------------------------------|
| GET    | <code>/config/providers`</code> | Active embedding + LLM providers (no secrets) |
| GET    | <code>/abbreviations`</code>    | Read the query-expansion dictionary           |

| PUT | `/abbreviations` | Update the dictionary |  
| GET | `/health/live` | Process liveness |  
| GET | `/health/ready` | Dependency readiness (DB, providers) |

#### ### 13.8 Future — Voice Extension (not built now)

| Method | Endpoint | Function |  
|---|---|---|  
| POST | `/voice/transcriptions` | Speech-to-text for a spoken query |  
| POST | `/voice/speech` | Text-to-speech for a response |

---

### ## 14. API Standards

- JSON request/response with a consistent error envelope: `{ "error": { "code", "message", "details" } }`.
- Input validation with Zod at the route boundary.
- `202 Accepted` + job ID for async operations (ingestion, generation, export).
- `X-Correlation-ID` accepted from the client or generated per request, and logged through the pipeline.
- Cursor or offset pagination for list endpoints.
- `projectId` scope enforced server-side, never widened by client input.

---

### ## 15. Security & Good Practices (Learner-appropriate)

- Keep API keys in `.env`; commit only `.env.example`; git-ignore `.env`.
- Validate all required env vars at startup and fail fast (typed config module).
- Validate uploads: allowed MIME types, extension, and a size limit.
- Treat retrieved content as untrusted data in prompts (separate instructions from evidence to reduce prompt injection).
- Do not log secrets or full sensitive prompts/responses; log by reference/IDs.
- `projectId` isolation enforced in the query layer.

> **Upgrade path:** SSO/RBAC, a secrets manager, redaction of sensitive data before external LLM calls, and full audit logging.

---

### ## 16. Observability

- Structured JSON logging (pino or winston).
- A correlation/request ID on every request, propagated to job and provider logs so one generation can be traced end-to-end.
- Health endpoints for liveness and dependency readiness.

> **Upgrade path (optional):** basic metrics (request latency, LLM token usage/cost per generation), then tracing/dashboards.

---

### ## 17. Reliability & Failure Handling

- Ingestion jobs are idempotent by `(projectId, sourceId, version)` + chunk `contentHash`.
- Bounded retries and timeouts on embedding and LLM calls.
- Failed generations are marked `failed` (never store invalid/unvalidated output).
- Partial ingestion results are reported with explicit warnings.

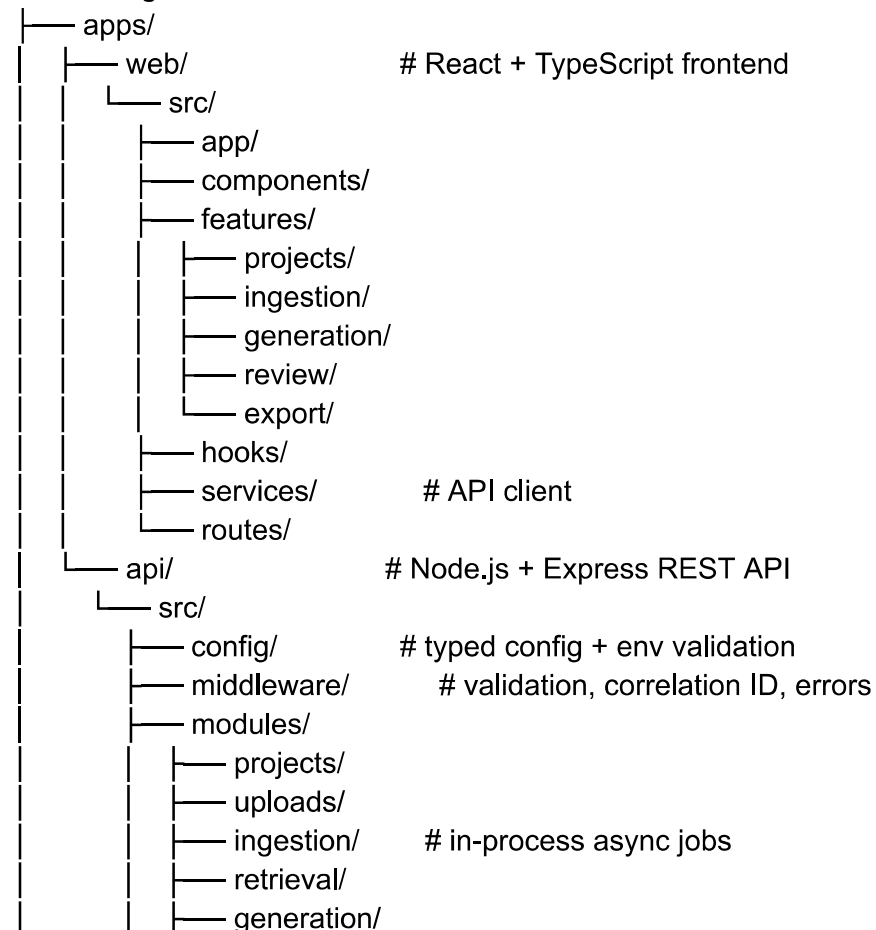
---

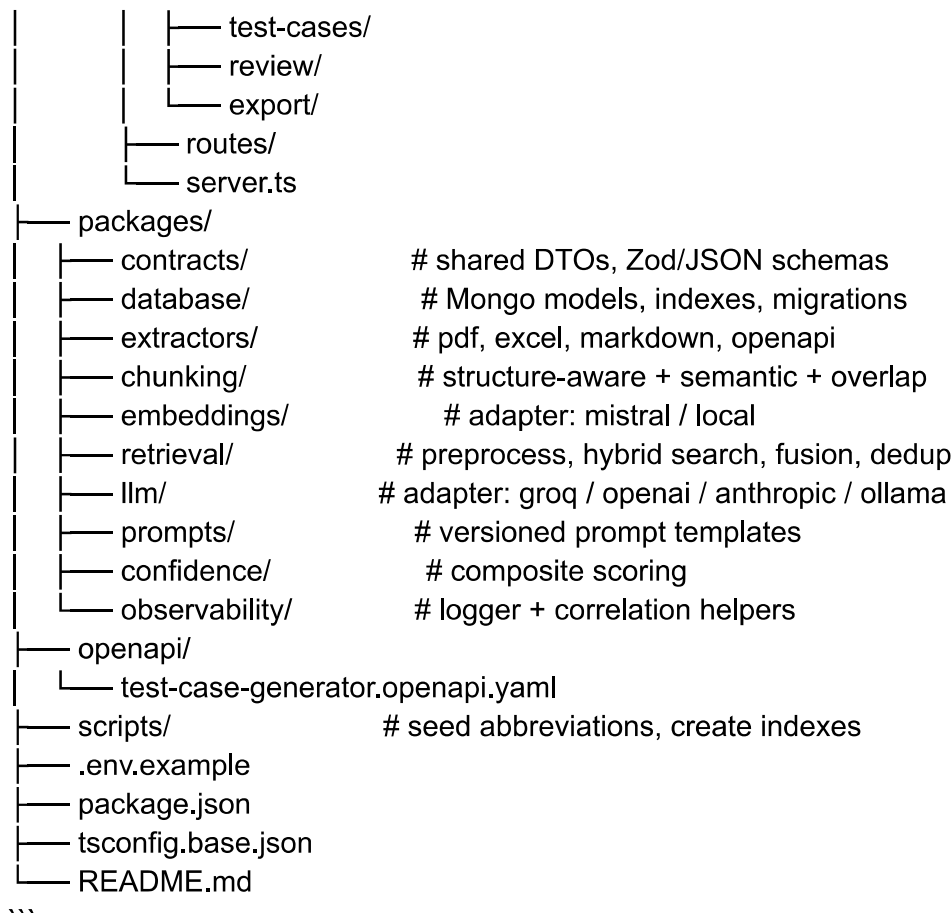
### ## 18. Project Folder Structure

A TypeScript monorepo keeps the frontend, API, and shared code aligned. It starts simple (one API process) but is organized so a worker/queue can be split out later.

```text

test-case-generator/





...

---

## ## 19. Architecture Acceptance Criteria

- A user can create a project and all data is scoped by `projectId`.
- A PDF/Excel/Markdown file and a Swagger spec can each be uploaded and ingested via an async job with a pollable status.
- Content is chunked with a structure-aware + semantic strategy and stored with embeddings.
- Retrieval combines BM25 and vector search, fuses scores, deduplicates, and preserves citations.
- Generation returns test cases that validate against the strict JSON Schema (with one repair retry).
- Each test case carries a composite confidence score/band and evidence citations.
- A draft test case can be edited, approved, or rejected with a comment.
- Approved/selected test cases export to JSON and CSV/Excel.
- Requests are traceable via a correlation ID in structured logs.
- Embedding and LLM providers can be swapped via config without changing pipeline code.

---

## ## 20. Open Decisions / Future Enhancements

1. Whether to add LLM-based query rewriting on top of dictionary expansion.
  2. When to add summarization/compression in context assembly (target end-state).
  3. When to add a reranking step after hybrid fusion.
  4. Whether to add a coverage matrix view (traceability C) on top of citations.
  5. Exact Mistral embedding model and its vector dimension (must match the index).
  6. If/when to move ingestion to a separate worker + queue (Pub/Sub / BullMQ).
  7. Voice scope (input only, or full two-way) if the extension is ever built.
- ...